

Three-way comparison: t1_t2_legacy vs maxeffCons vs maxcons

M1, target HR harm = 1.0, $n = 500$, 500 replicates

2026-08-03

```
data_dir <- "."

arm_files <- c(
  t1_t2_legacy = "fs_t1_t2_m1_h10_knoise0_n500_combined_1_500.rds",
  maxeffCons   = "fs_maxeffCons_fb_mr_m1_h10_knoise0_n500_combined_1_500.rds",
  maxcons      = "fs_maxcons_fb_mr_m1_h10_knoise0_n500_combined_1_500.rds"
)

#' Load simulation bundles and resolve the legacy/new column naming.
#'
#' Legacy runs record the full bootstrap as `t1_*` and multiplier resampling as
#' `t2_*`; the revised codebase records them as `fb_*` and `mr_*`.
load_arms <- function(data_dir, files = arm_files) {
  arms <- lapply(names(files), function(nm) {
    bundle <- readRDS(file.path(data_dir, files[[nm]]))
    legacy <- "t1_H_est" %in% names(bundle$results)
    list(
      name      = nm,
      results   = bundle$results,
      truth     = bundle$truth,
      meta      = bundle$meta,
      fb        = if (legacy) "t1" else "fb",
      mr        = if (legacy) "t2" else "mr",
      harm_flag = if (legacy) "t2_harm_flag" else "mr_harm_flag",
      mr_ok     = if (legacy) "gate_ok" else "mr_ok",
      fb_secs   = if (legacy) "t1_secs" else "fb_secs",
      mr_secs   = if (legacy) "t2_secs" else "fit_mr_secs"
    )
  })
  names(arms) <- names(files)
  arms
}
```

```

detected_of <- function(arm) {
  arm$results[arm$results$detected == 1, , drop = FALSE]
}

num <- function(x, digits = 3) formatC(x, format = "f", digits = digits)

arm_table <- function(arms, rows) {
  body <- lapply(rows, function(f) vapply(arms, f, character(1)))
  df <- as.data.frame(do.call(rbind, body), stringsAsFactors = FALSE)
  cbind(Metric = names(rows), df, row.names = NULL)
}

show_table <- function(df, caption) {
  knitr::kable(df, booktabs = TRUE, linesep = "", align = "l",
    escape = FALSE, caption = caption)
}

#' Align two arms on `sim_id` so replicates can be compared pairwise.
align_on_sim_id <- function(arm_a, arm_b) {
  key <- arm_a$results$sim_id
  list(a = arm_a$results,
    b = arm_b$results[match(key, arm_b$results$sim_id), , drop = FALSE])
}

arms <- load_arms(data_dir)

```

All three bundles share the design cell: `n_sample` 500, `n_sims` 500, `nb_boots` 300, `draws` 5000, `seed_base` 8316951, `consistency_method` = "resample", `forestsearch` 0.2.0, and identical truth. The arms differ only in `sg_focus`.

```

design <- data.frame(
  arm      = names(arms),
  sg_focus = vapply(arms, function(a) {
    if (is.null(a$meta$sg_focus)) "eff (legacy)" else a$meta$sg_focus
  }, character(1)),
  n_sample = vapply(arms, function(a) a$meta$n_sample, integer(1)),
  nb_boots = vapply(arms, function(a) a$meta$nb_boots, integer(1)),
  seed_base = vapply(arms, function(a) a$meta$seed_base, integer(1)),
  batches   = vapply(arms, function(a) a$meta$n_batches, integer(1)),
  row.names = NULL
)
show_table(design, "Design constants and provenance.")
stopifnot(
  all.equal(arms$t1_t2_legacy$truth, arms$maxeffCons$truth),
  all.equal(arms$t1_t2_legacy$truth, arms$maxcons$truth)
)

```

Table 1: Design constants and provenance.

arm	sg_focus	n_sample	nb_boots	seed_base	batches
t1_t2_legacy	eff (legacy)	500	300	8316951	4
maxeffCons	maxeffCons	500	300	8316951	3
maxcons	maxcons	500	300	8316951	3

1 Detection

Detection agrees replicate by replicate across all three arms.

```
detection_cross <- function(arm_a, arm_b) {
  al <- align_on_sim_id(arm_a, arm_b)
  sum(al$a$detected == al$b$detected)
}

pairs <- list(
  c("t1_t2_legacy", "maxeffCons"),
  c("t1_t2_legacy", "maxcons"),
  c("maxeffCons", "maxcons")
)

detection <- data.frame(
  comparison = vapply(pairs, function(p) paste(p, collapse = " vs "), character(1)),
  agree      = vapply(pairs, function(p) {
    detection_cross(arms[[p[1]]], arms[[p[2]]])
  }, integer(1)),
  of         = nrow(arms$t1_t2_legacy$results),
  row.names  = NULL
)
show_table(detection, "Replicate-level detection agreement.")
vapply(arms, function(a) sum(a$results$detected), integer(1))
```

t1_t2_legacy	maxeffCons	maxcons
346	346	346

Table 2: Replicate-level detection agreement.

comparison	agree	of
t1_t2_legacy vs maxeffCons	500	500
t1_t2_legacy vs maxcons	500	500
maxeffCons vs maxcons	500	500

2 Selection

```
#' Share of jointly detected replicates selecting the identical rule.
selection_agreement <- function(arm_a, arm_b) {
  al <- align_on_sim_id(arm_a, arm_b)
  det <- al$a$detected == 1 & al$b$detected == 1
  same <- det & al$a$label == al$b$label
  c(detected = sum(det, na.rm = TRUE),
    same      = sum(same, na.rm = TRUE),
    pct       = 100 * sum(same, na.rm = TRUE) / sum(det, na.rm = TRUE))
}

agreement <- do.call(rbind, lapply(pairs, function(p) {
  s <- selection_agreement(arms[[p[1]]], arms[[p[2]]])
  data.frame(comparison = paste(p, collapse = " vs "),
    detected   = s[["detected"]],
    same_rule  = s[["same"]],
    pct        = num(s[["pct"]], 1),
    row.names  = NULL)
}))
show_table(agreement, "Selection agreement on jointly detected replicates.")
```

Table 3: Selection agreement on jointly detected replicates.

comparison	detected	same_rule	pct
t1_t2_legacy vs maxeffCons	346	281	81.2
t1_t2_legacy vs maxcons	346	283	81.8
maxeffCons vs maxcons	346	246	71.1

```
rows_size <- list(
  "mean  $\widehat{H}$ " = function(a) num(mean(detected_of(a)$n_harm), 1),
  "SD  $\widehat{H}$ " = function(a) num(sd(detected_of(a)$n_harm), 1),
  "median  $\widehat{H}$ " = function(a) num(median(detected_of(a)$n_harm), 0),
  "min  $\widehat{H}$ " = function(a) num(min(detected_of(a)$n_harm), 0),
  "max  $\widehat{H}$ " = function(a) num(max(detected_of(a)$n_harm), 0),
  "Sensitivity" = function(a) num(mean(detected_of(a)$sens)),
  "Specificity" = function(a) num(mean(detected_of(a)$spec)),
  "PPV" = function(a) num(mean(detected_of(a)$ppv)),
  "NPV" = function(a) num(mean(detected_of(a)$npv)),
  "mean  $\beta(\widehat{H})$ " = function(a) num(mean(detected_of(a)$betaHhat_H))
)
show_table(arm_table(arms, rows_size),
  "Subgroup size and classification, detected replicates.")
```

Table 4: Subgroup size and classification, detected replicates.

Metric	t1_t2_legacy	maxeffCons	maxcons
mean $ \widehat{H} $	82.3	78.1	87.5
SD $ \widehat{H} $	21.8	17.8	28.1
median $ \widehat{H} $	76	73	79
min $ \widehat{H} $	61	61	61
max $ \widehat{H} $	184	167	220
Sensitivity	0.390	0.368	0.405
Specificity	0.867	0.874	0.858
PPV	0.315	0.309	0.309
NPV	0.909	0.907	0.910
mean $\beta(\widehat{H})$	0.728	0.726	0.725

3 MR at coincident selection

The selection map $\mathcal{S}$ enters the correction through the per-draw re-selected winner $\widehat{H}_b^*$, so arms with different $\mathcal{S}$ are expected to differ in MR even where the observed winner coincides. The table below restricts to replicates on which the two arms selected the identical rule.

```
#' Compare MR and FB output on replicates where both arms selected the same
#' rule, so that any difference is attributable to the procedure rather than to
#' the region being estimated.
coincident_selection_diff <- function(arm_a, arm_b) {
  al <- align_on_sim_id(arm_a, arm_b)
  same <- al$a$detected == 1 & al$b$detected == 1 & al$a$label == al$b$label
  same[is.na(same)] <- FALSE

  pull <- function(d, prefix, suffix) d[[paste0(prefix, "_H_", suffix)]] [same]
  mr_d <- pull(al$a, arm_a$mr, "est") - pull(al$b, arm_b$mr, "est")
  hi_d <- pull(al$a, arm_a$mr, "hi") - pull(al$b, arm_b$mr, "hi")
  fb_d <- pull(al$a, arm_a$fb, "est") - pull(al$b, arm_b$fb, "est")
  se_a <- al$a[[paste0(arm_a$mr, "_H_se_ij")]] [same]
  se_b <- al$b[[paste0(arm_b$mr, "_H_se_ij")]] [same]

  data.frame(
    n = sum(same),
    mr_max_abs = max(abs(mr_d), na.rm = TRUE),
    mr_hi_max_abs = max(abs(hi_d), na.rm = TRUE),
    mr_mean_diff = mean(mr_d, na.rm = TRUE),
    mr_n_exact = sum(abs(mr_d) < 1e-10, na.rm = TRUE),
    mr_n_gt_0.01 = sum(abs(mr_d) > 0.01, na.rm = TRUE),
    mr_n_same_sign = sum(mr_d > 0, na.rm = TRUE),
    se_mean_a = mean(se_a, na.rm = TRUE),
    se_mean_b = mean(se_b, na.rm = TRUE),
```

```

    fb_mean_abs      = mean(abs(fb_d), na.rm = TRUE),
    row.names        = NULL
  )
}

coincident <- do.call(rbind, lapply(c("maxcons", "maxeffCons"), function(nm) {
  cbind(comparison = paste("legacy vs", nm),
        coincident_selection_diff(arms$t1_t2_legacy, arms[[nm]]))
}))

coincident_fmt <- coincident
for (j in c("mr_max_abs", "mr_hi_max_abs", "mr_mean_diff")) {
  coincident_fmt[[j]] <- formatC(coincident[[j]], format = "g", digits = 3)
}
for (j in c("se_mean_a", "se_mean_b", "fb_mean_abs")) {
  coincident_fmt[[j]] <- num(coincident[[j]], 4)
}
show_table(coincident_fmt,
           "MR and FB agreement restricted to identical-selection replicates.")

```

Table 5: MR and FB agreement restricted to identical-selection replicates.

comparison	n	mr_n_ex-				mr_n_gt_0.01	n_0.01_same_sign	se_mean_a	fb_bmean_abs
		mr_max_abs	mr_hi_max_abs	mr_mean_diff	mr_n				
legacy vs maxcons	283	6.66e-16	3.55e-15	4.32e-18	283	0	14	0.3519	0.3519
legacy vs maxeffCons	281	0.285	0.936	0.067	0	267	247	0.3627	0.3863
								0.0145	0.0036

4 Estimation and coverage

```

est_summary <- function(arm, prefix, block = "H") {
  d <- detected_of(arm)
  est <- d[[paste0(prefix, "_", block, "_est")]]
  lo <- d[[paste0(prefix, "_", block, "_lo")]]
  hi <- d[[paste0(prefix, "_", block, "_hi")]]
  tgt <- d[[paste0("betaHhat_", block)]]
  ok <- is.finite(est) & is.finite(lo) & is.finite(hi) & is.finite(tgt)
  c(
    avg = mean(est[ok]),
    len = median((hi - lo)[ok]),
    bias = 100 * mean((est[ok] - tgt[ok]) / tgt[ok]),
    cov = mean(tgt[ok] >= lo[ok] & tgt[ok] <= hi[ok])
  )
}

```

```

}

prefix_for <- function(arm, estimator) {
  switch(estimator, oracle = "or", naive = "nv", FB = arm$fb, MR = arm$mr)
}

estimation_table <- function(arms, block = "H",
                             estimators = c("oracle", "naive", "FB", "MR")) {
  metrics <- c(avg = "Avg", len = "Len", bias = "Bias (\\%)", cov = "Cov")
  parts <- lapply(estimators, function(e) {
    vals <- lapply(arms, function(a) est_summary(a, prefix_for(a, e), block))
    tab <- vapply(names(metrics), function(m) {
      digits <- if (m == "bias") 1 else 3
      vapply(vals, function(v) num(v[[m]], digits), character(1))
    }, character(length(arms)))
    data.frame(
      Estimator = c(e, rep("", length(metrics) - 1L)),
      Metric = unname(metrics),
      t(tab),
      check.names = FALSE, row.names = NULL, stringsAsFactors = FALSE
    )
  })
  do.call(rbind, parts)
}

show_table(estimation_table(arms, "H"),
            "Harm block $\\widehat{H}$, against $\\beta(\\widehat{H})$.")
show_table(estimation_table(arms, "Hc"),
            "Complement block $\\widehat{H}^{\\mathrm{c}}$.")

```

Table 6: Complement block $\widehat{H}^c$.

Estimator	Metric	t1_t2_legacy	maxeffCons	maxcons
oracle	Avg	1.135	1.135	1.135
	Len	1.441	1.441	1.441
	Bias (%)	54.9	55.2	55.6
	Cov	0.835	0.838	0.827
naive	Avg	1.629	1.643	1.611
	Len	2.082	2.157	1.964
	Bias (%)	128.0	130.9	125.9
	Cov	0.145	0.153	0.130
FB	Avg	1.222	1.229	1.223
	Len	1.659	1.749	1.648
	Bias (%)	71.0	72.6	71.3
	Cov	0.801	0.806	0.783
MR	Avg	1.003	0.946	0.992

Table 6: Complement block $\widehat{H}^c$.

Estimator	Metric	t1_t2_legacy	maxeffCons	maxcons
	Len	1.473	1.536	1.395
	Bias (%)	40.0	32.8	38.7
	Cov	0.968	0.986	0.968

Table 7: Complement block $\widehat{H}^c$.

Estimator	Metric	t1_t2_legacy	maxeffCons	maxcons
oracle	Avg	0.660	0.660	0.660
	Len	0.360	0.360	0.360
	Bias (%)	0.1	-0.1	0.1
	Cov	0.960	0.960	0.960
naive	Avg	0.605	0.609	0.598
	Len	0.335	0.336	0.336
	Bias (%)	-8.2	-7.6	-9.2
	Cov	0.916	0.928	0.908
FB	Avg	0.631	0.636	0.626
	Len	0.490	0.487	0.485
	Bias (%)	-4.2	-3.6	-5.0
	Cov	0.983	0.988	0.986
MR	Avg	0.665	0.655	0.658
	Len	0.672	0.679	0.667
	Bias (%)	0.9	-0.7	-0.2
	Cov	1.000	1.000	1.000

The oracle refits on the true region H , so its Bias% and Cov entries are computed against a different region from the one it estimates.

5 Timings

```
rows_timing <- list(
  "FB mean (s)"      = function(a) num(mean(detected_of(a)[[a$fb_secs]], na.rm = TRUE), 1),
  "FB median (s)"    = function(a) num(median(detected_of(a)[[a$fb_secs]], na.rm = TRUE), 1),
  "FB min (s)"       = function(a) num(min(detected_of(a)[[a$fb_secs]], na.rm = TRUE), 1),
  "FB max (s)"       = function(a) num(max(detected_of(a)[[a$fb_secs]], na.rm = TRUE), 1),
  "fit+MR mean (s)"  = function(a) num(mean(a$results[[a$mr_secs]], na.rm = TRUE), 2),
  "fit+MR median (s)" = function(a) num(median(a$results[[a$mr_secs]], na.rm = TRUE), 2),
  "fit+MR min (s)"   = function(a) num(min(a$results[[a$mr_secs]], na.rm = TRUE), 2),
  "fit+MR max (s)"   = function(a) num(max(a$results[[a$mr_secs]], na.rm = TRUE), 2)
)
```

```
show_table(arm_table(arms, rows_timing),
           "Wall-clock seconds per replicate under 115 workers.")
```

Table 8: Wall-clock seconds per replicate under 115 workers.

Metric	t1_t2_legacy	maxeffCons	maxcons
FB mean (s)	166.4	136.2	167.5
FB median (s)	117.2	135.2	185.4
FB min (s)	100.8	105.2	113.3
FB max (s)	397.6	165.3	192.1
fit+MR mean (s)	12.78	5.01	12.66
fit+MR median (s)	11.38	5.41	10.99
fit+MR min (s)	3.37	2.14	3.33
fit+MR max (s)	23.64	6.97	23.47

Per-replicate cost varies by batch within each arm, so cross-arm differences reflect render conditions as well as any code effect.

```
#' Mean seconds per replicate within `sim_id` segments. Batch boundaries are not
#' stored numerically in `meta`, so they are read off `meta$source_files`.
timing_by_batch <- function(arm, breaks, secs = c("fb", "mr")) {
  secs <- match.arg(secs)
  d <- arm$results
  keep <- if (secs == "fb") d$detected == 1 else rep(TRUE, nrow(d))
  col <- if (secs == "fb") arm$fb_secs else arm$mr_secs
  seg <- cut(d$sim_id, breaks = breaks)
  round(tapply(d[[col]][keep], seg[keep], mean, na.rm = TRUE), 2)
}

batch_breaks <- list(
  t1_t2_legacy = c(0, 20, 100, 200, 500),
  maxeffCons   = c(0, 50, 300, 500),
  maxcons      = c(0, 20, 300, 500)
)

batch_timing <- do.call(rbind, lapply(names(arms), function(nm) {
  data.frame(
    arm      = nm,
    FB       = paste(timing_by_batch(arms[[nm]], batch_breaks[[nm]], "fb"),
                     collapse = " / "),
    fit_MR   = paste(timing_by_batch(arms[[nm]], batch_breaks[[nm]], "mr"),
                     collapse = " / "),
    row.names = NULL
  )
}))
```

```
show_table(batch_timing, "Mean seconds per replicate, by batch.")
#' Correlation of per-replicate cost with selected subgroup size.
vapply(arms, function(a) {
  d <- detected_of(a)
  cor(d[[a$fb_secs]], d$n_harm, use = "complete.obs")
}, numeric(1))
```

```
t1_t2_legacy    maxeffCons    maxcons
-0.0001856616 -0.1076099258  0.0250748790
```

Table 9: Mean seconds per replicate, by batch.

arm	FB	fit_MR
t1_t2_legacy	113.29 / 372.26 / 179.83 / 114.46	15.7 / 9.11 / 5.02 / 16.15
maxeffCons	132.76 / 131.67 / 142.92	4.85 / 4.92 / 5.16
maxcons	187 / 187.85 / 136.46	16.57 / 17.85 / 5.02

6 Session information

```
sessionInfo()
```

R version 4.6.1 (2026-06-24)

Platform: x86_64-pc-linux-gnu

Running under: Pop!_OS 24.04 LTS

Matrix products: default

BLAS: /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0

LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0 LAPACK version 3.12.0

locale:

```
[1] LC_CTYPE=en_US.UTF-8      LC_NUMERIC=C
[3] LC_TIME=en_US.UTF-8      LC_COLLATE=en_US.UTF-8
[5] LC_MONETARY=en_US.UTF-8  LC_MESSAGES=en_US.UTF-8
[7] LC_PAPER=en_US.UTF-8     LC_NAME=C
[9] LC_ADDRESS=C             LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
```

time zone: America/Los_Angeles

tzcode source: system (glibc)

attached base packages:

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

loaded via a namespace (and not attached):

```
[1] compiler_4.6.1    fastmap_1.2.0    cli_3.6.6        tools_4.6.1
[5] htmltools_0.5.9  otel_0.2.0       rstudioapi_0.19.0 yaml_2.3.12
[9] rmarkdown_2.31   knitr_1.51       jsonlite_2.0.0   xfun_0.60
[13] digest_0.6.39    rlang_1.3.0      evaluate_1.0.5
```